Lab 2 — Part 4: Data Analytics with R and ClickHouse

From Data Pipeline to Business Insights — v1.1.0

Allan Omondi

19 June 2026

Table of contents

1 Introduction **2**

2 Setup: Load Libraries and Connect **2**

3 Exploratory Data Analysis **3**
- 3.1 Overview of the Orders Data . . . 3
- 3.2 Order Distribution by Item . . . 6
- 3.3 Bulk versus Standard Order Breakdown . . . 8
- 3.4 Order Volume Over Time . . . 10

4 Hypothesis Tests **12**
- 4.1 Hypothesis Test 1: Do Bulk Orders Differ in Quantity? . . . 13
- 4.2 Hypothesis Test 2: Is Order Quantity Independent of Item Type? . . . 17

5 The Performance Benchmark **20**
- 5.1 The "Cold Start" Problem . . . 20
  - 5.1.1 The Concept: Cold versus Warm . . . 20
  - 5.1.2 Library Analogy . . . 20
  - 5.1.3 Why it Matters for Benchmarking . . . 21
  - 5.1.4 Strategic Counter-Position . . . 21
- 5.2 The Query . . . 21
- 5.3 Run the Benchmark . . . 22
- 5.4 Visualise the Results . . . 24
- 5.5 Examine the Query Results . . . 26
- 5.6 Why the Gap Exists: Column Store vs Row Store . . . 28

6 Data Pipeline Latency Analysis 29

7 Conclusion 34

8 Clean Up 34

1 Introduction

In Parts 1 through 3 we built a complete data engineering pipeline that moves orders from a producer, through Kafka, into PostgreSQL, and then into ClickHouse via Debezium CDC and a custom transformer service coded using Python.

In this notebook you will use R to:

1. Connect to both databases and inspect the data
2. Perform analytical queries to answer business questions
3. Run the **same query** on PostgreSQL and ClickHouse and measure the performance difference
4. Visualise the results using `ggplot2`

The central argument (the thesis) of this lab is simple: **the data pipeline we built exists because analytical queries do not belong in the operational data store (ACID-compliant relational databases designed to support database transactions).** By the end of this notebook, you will:

- **Measure** the performance difference between executing an analytical query in an operational data store (relational [object relational] PostgreSQL database) versus in an analytical data store (ClickHouse non-relational [column-oriented] data warehouse).

---

2 Setup: Load Libraries and Connect

```
#> Current working directory: C:/Users/aomondi/Documents/GitHub/Teaching/Kafka/4_data_analyti
#> [1] "connect_clickhouse_with_ODBC.R"
#> [2] "generate_data.py"
#> [3] "instructions_for_project_setup.md"
#> [4] "lab4_analytics_with_odbc_quarto.docx"
#> [5] "lab4_analytics_with_odbc_quarto.html"
#> [6] "lab4_analytics_with_odbc_quarto.pdf"
#> [7] "lab4_analytics_with_odbc_quarto.qmd"
#> [8] "lab4_analytics_with_odbc_quarto.rmarkdown"
```

We start by connecting to both PostgreSQL and ClickHouse using the ODBC drivers. The output confirms that the connection is successful

```
#> ch_conn (ClickHouse) -> connected and responsive
#> pg_conn (PostgreSQL) -> connected and responsive
```

````
```{r confirm_connections}
# These connections must already exist from the previous chunk.
stopifnot(
  "ch_conn not found - run connect_clickhouse_with_ODBC.R first" = exists("ch_conn"),
  "pg_conn not found - run connect_clickhouse_with_ODBC.R first" = exists("pg_conn")
)

# Confirm row counts
ch_count <- dbGetQuery(ch_conn, "SELECT COUNT(*) AS n FROM orders")$n
pg_count <- dbGetQuery(pg_conn, "SELECT COUNT(*) AS n FROM orders")$n

cat(sprintf("ClickHouse : %s rows\n", format(as.numeric(ch_count), big.mark = ",")))
cat(sprintf("PostgreSQL : %s rows\n", format(as.numeric(pg_count), big.mark = ",")))
```
````

```
#> ClickHouse : 503,262 rows
#> PostgreSQL : 503,262 rows
```

---

3 Exploratory Data Analysis

The first step in any data-related project is to understand the structure and distribution of the data before running statistical tests, training models, solving optimization problems, creating dashboards, etc. This section demonstrates that workflow using the ClickHouse data warehouse.

3.1 Overview of the Orders Data

While `ROUND()` is excellent for final reports, it is often a mistake to apply rounding within a query **if** those values are to be used for further calculations later. Rounding early introduces "truncation error" that compounds as you aggregate further. It is technically superior to keep

the full precision in the database and handle the visual formatting (the "two decimal places") within the data analytics reporting layer itself using functions like `round()` or `sprintf()`.

As noted in Part 3 of the lab, the timezone used in the PostgreSQL database is UTC+2 (Lagos, Nigeria) then the transformer converts this so that by the time it is stored in the ClickHouse data warehouse, it is stored as UTC+3 (Nairobi, Kenya).

In other words, consider a multinational business. PostgreSQL is in a branch in Lagos, Nigeria while ClickHouse is in the Headquarters in Nairobi, Kenya.

So the data retrieved from ClickHouse is already in UTC+3 or East African Time (EAT) thanks to the work done by the transformer.

```
```{r eda-overview}
# Pull a representative sample for local exploration.
# Note: for summarising the full dataset, we push computation to ClickHouse
# using SQL and only bring the result into R. This is best practice -
# never pull millions of rows into R when a database can compute the
# summary for you. Otherwise, it beats the purpose of the business investing in
# acquiring a powerful database server or data warehouse or data lake.
overview <- dbGetQuery(ch_conn, "
  SELECT
    COUNT(*)                                AS total_orders,
    COUNT(DISTINCT customer_name)           AS unique_customers,
    COUNT(DISTINCT item)                    AS unique_items,
    SUM(order_quantity)                     AS total_units,
    ROUND(AVG(order_quantity), 2)           AS avg_quantity_per_order,
    SUM(is_bulk_order)                      AS bulk_orders,
    ROUND(SUM(is_bulk_order) / COUNT(*) * 100, 2) AS bulk_order_pct,
    toTimeZone(MIN(received_at), 'Africa/Nairobi') AS earliest_order,
    toTimeZone(MAX(received_at), 'Africa/Nairobi') AS latest_order
  FROM orders
  WHERE operation != 'DELETE'
")

overview$total_orders            <- as.integer(overview$total_orders)
overview$unique_customers        <- as.integer(overview$unique_customers)
overview$unique_items            <- as.integer(overview$unique_items)
overview$total_units             <- as.integer(overview$total_units)
overview$avg_quantity_per_order  <- as.numeric(overview$avg_quantity_per_order)
overview$bulk_orders             <- as.integer(overview$bulk_orders)
overview$bulk_order_pct          <- as.numeric(overview$bulk_order_pct)

overview$earliest_order <- as.POSIXct(overview$earliest_order, tz = "UTC")
```
```

```r
overview$latest_order   <- as.POSIXct(overview$latest_order, tz = "UTC")

overview$earliest_order <- format(
  overview$earliest_order,
  "%Y-%m-%d %H:%M:%S %Z",
  tz = "Africa/Nairobi"
)

overview$latest_order <- format(
  overview$latest_order,
  "%Y-%m-%d %H:%M:%S %Z",
  tz = "Africa/Nairobi"
)

overview_t <- data.frame(
  Metric = names(overview),
  # I() stands for "inhibit interpretation." It means "store this as-is,
  # without simplification."

  # Without I(), data.frame() would try to expand as.list(overview[1, ]) - a list -
  # into separate columns, one per element. I() tells data.frame() to treat the
  # entire list as a single column where each cell holds one list element.
  Value  = I(as.list(overview[1, ]))
)

overview_t$Value <- sapply(overview_t$Value, function(x) {
  if (inherits(x, "POSIXct")) {
    format(x, "%Y-%m-%d %H:%M:%S %Z")
  } else if (is.numeric(x)) {
    format(as.numeric(x), scientific = FALSE, big.mark = ",")
  } else {
    as.character(x)
  }
})

print(overview_t)
```

```
#>                                    Metric                   Value
#> total_orders                 total_orders                 503,262
#> unique_customers       unique_customers                      46
#> unique_items                 unique_items                      17
```

```
#> total_units                          total_units               5,266,725
#> avg_quantity_per_order avg_quantity_per_order                   10.47
#> bulk_orders                          bulk_orders                376,488
#> bulk_order_pct                    bulk_order_pct                  74.81
#> earliest_order                    earliest_order 2026-03-21 13:56:49 EAT
#> latest_order                        latest_order 2026-06-19 13:56:34 EAT
```

Reading the table: The `bulk_order_pct` shows what proportion of all orders are classified as bulk (quantity > 5). The gap between `earliest_order` and `latest_order` shows the time window covered by the dataset. These are possible first questions a data analyst can ask when handed a new dataset.

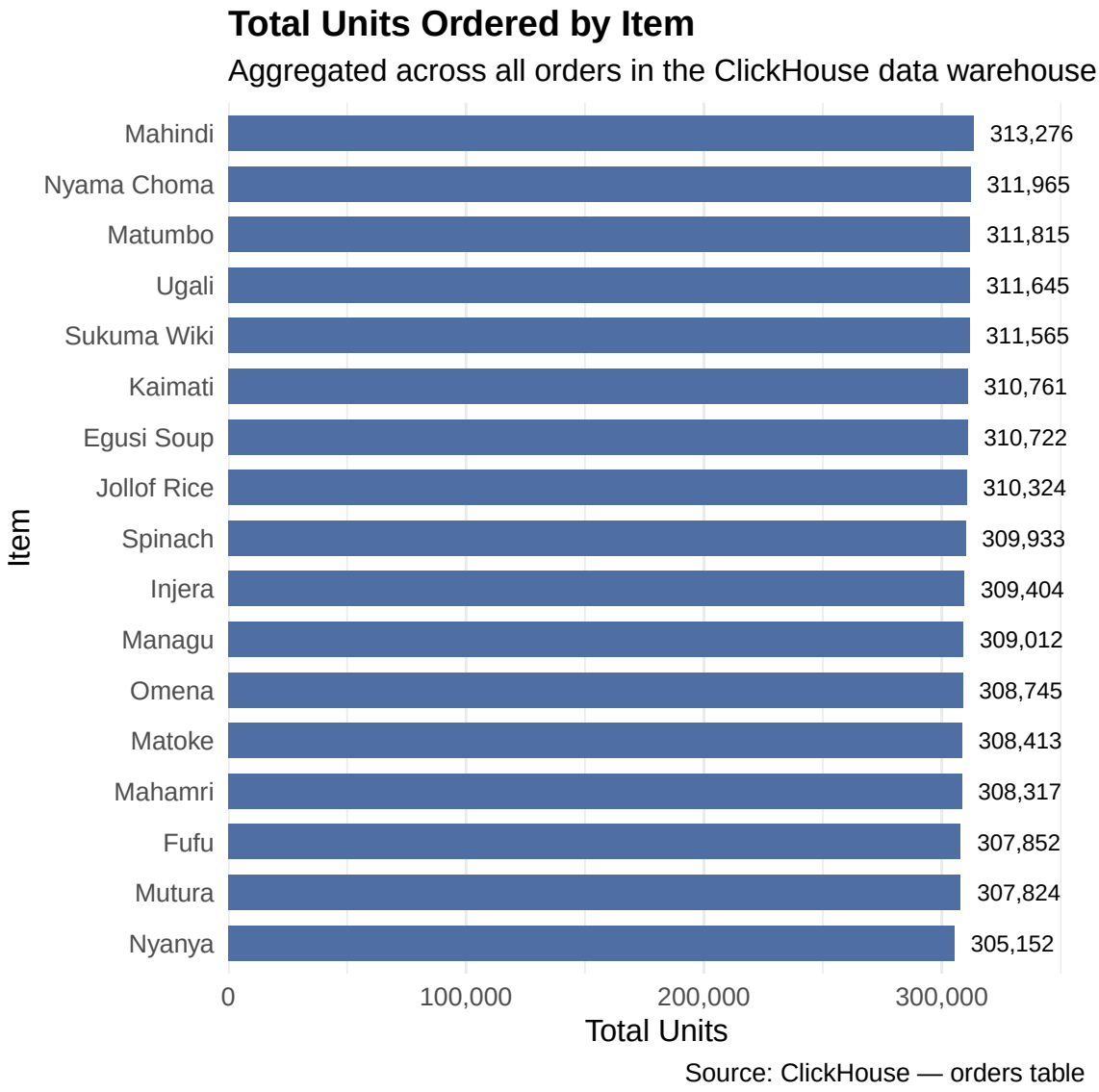
3.2 Order Distribution by Item

```
```{r orders-by-item}
item_summary <- dbGetQuery(ch_conn, "
  SELECT
    item,
    COUNT(*)          AS order_count,
    SUM(order_quantity) AS total_units,
    AVG(order_quantity) AS avg_quantity,
    SUM(is_bulk_order)  AS bulk_orders
  FROM orders
  WHERE operation != 'DELETE'
  GROUP BY item
  ORDER BY total_units DESC
")

pacman::p_load("ggplot2")
pacman::p_load("scales")

# `x = reorder(item, total_units)` Reorders items by total_units
# The default is to order in alphabetical order which is rarely useful for analysis
ggplot(item_summary, aes(x = reorder(item, total_units), y = total_units)) +
  geom_col(fill = "#4F6EA4", width = 0.7) +
  geom_text(
    # Explicitly cast to numeric to ensure format() recognizes the number
    aes(label = format(as.numeric(total_units), big.mark = ",")),
    hjust = -0.2,
    size = 3.2
```
```

```
  ) +
  # `coord_flip()` converts the standard vertical bar chart to a horizontal bar chart
  coord_flip() +
  scale_y_continuous(
    labels = label_comma(),
    expand = expansion(mult = c(0, 0.15))
  ) +
  labs(
    title    = "Total Units Ordered by Item",
    subtitle = "Aggregated across all orders in the ClickHouse data warehouse",
    x        = "Item",
    y        = "Total Units",
    caption  = "Source: ClickHouse - orders table"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title         = element_text(face = "bold"),
    panel.grid.major.y = element_blank()
  )
```



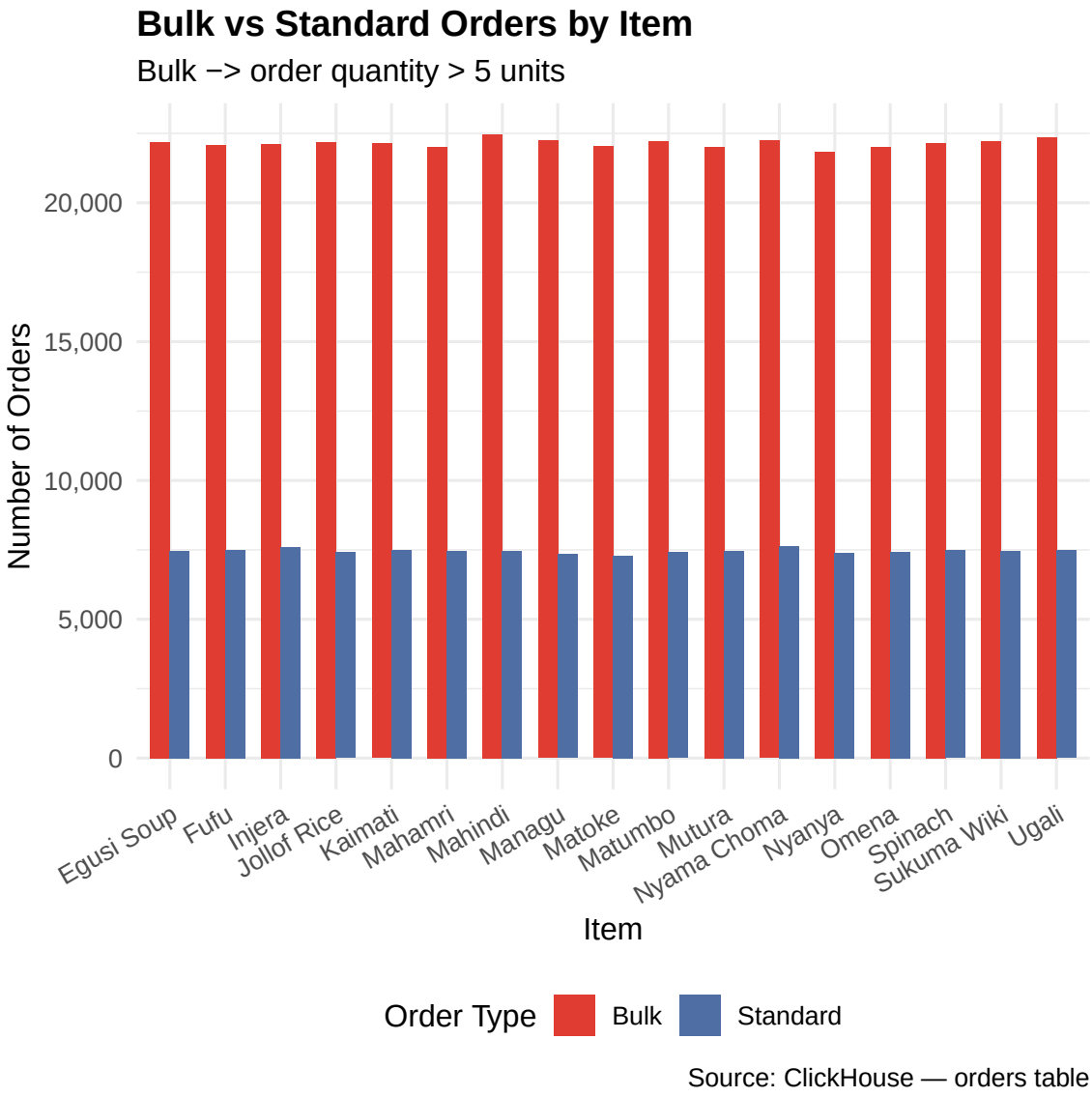
3.3 Bulk versus Standard Order Breakdown

```
```{r bulk-vs-standard}
bulk_summary <- dbGetQuery(ch_conn, "
  SELECT
    item,
    SUM(CASE WHEN is_bulk_order = 1 THEN 1 ELSE 0 END) AS bulk,
    SUM(CASE WHEN is_bulk_order = 0 THEN 1 ELSE 0 END) AS standard
```
```

```
  FROM orders
  WHERE operation != 'DELETE'
  GROUP BY item
  ORDER BY item
")

# Reshape to long format for grouped bar chart
bulk_long <- bulk_summary %>%
  pivot_longer(cols = c(bulk, standard),
               names_to  = "order_type",
               values_to = "count") %>%
  mutate(order_type = str_to_title(order_type))

ggplot(bulk_long, aes(x = reorder(item, -count), y = count, fill = order_type)) +
  geom_col(position = "dodge", width = 0.7) +
  scale_fill_manual(
    values = c("Bulk" = "#E03C31", "Standard" = "#4F6EA4"),
    name   = "Order Type"
  ) +
  scale_y_continuous(labels = comma) +
  labs(
    title    = "Bulk vs Standard Orders by Item",
    subtitle = "Bulk → order quantity > 5 units",
    x        = "Item",
    y        = "Number of Orders",
    caption  = "Source: ClickHouse - orders table"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title         = element_text(face = "bold"),
    legend.position    = "bottom",
    axis.text.x        = element_text(angle = 30, hjust = 1)
  )
```



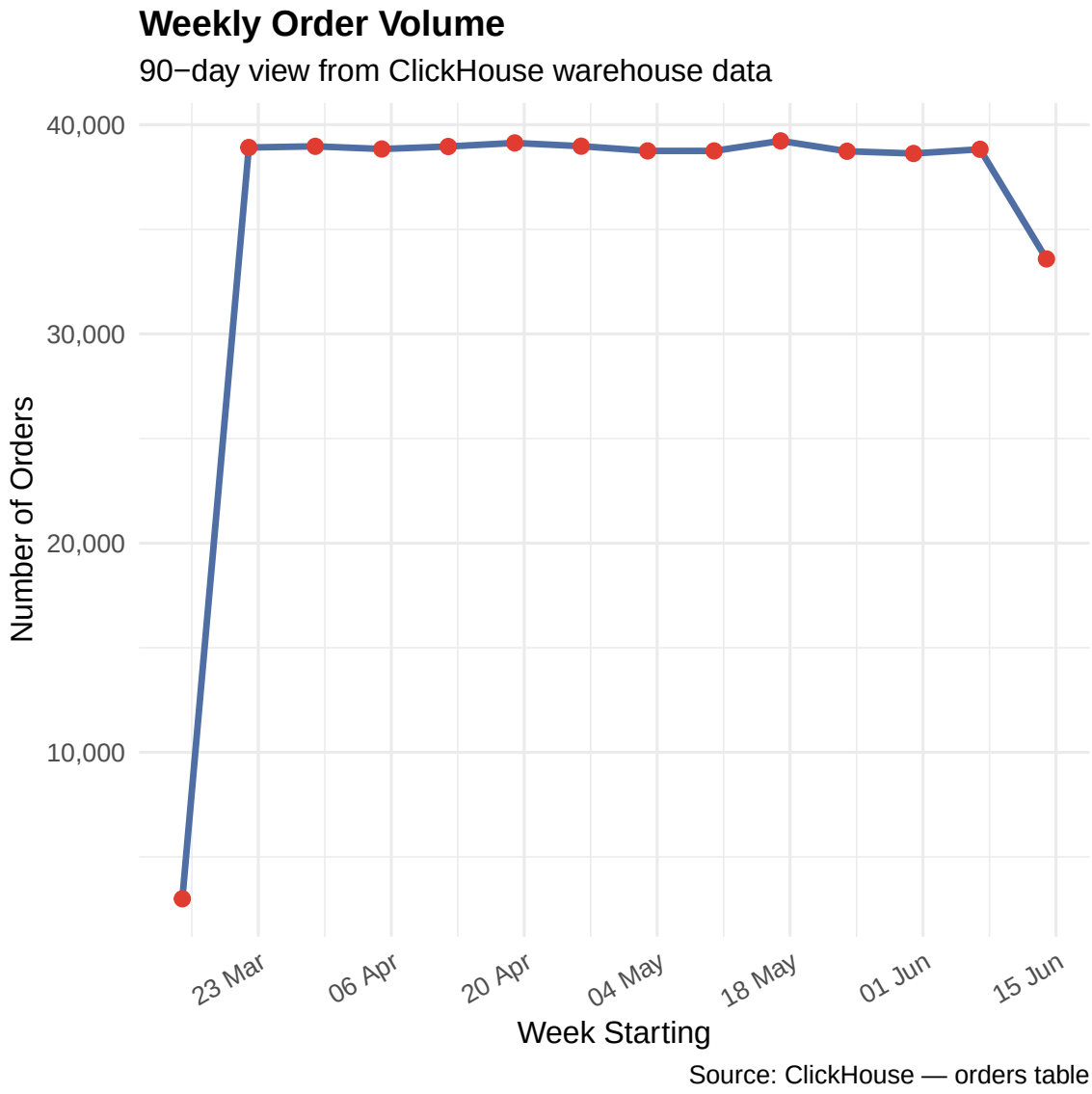
3.4 Order Volume Over Time

```
```{r orders-over-time}
# Aggregate orders by week to show trends over the 90-day period
weekly_orders <- dbGetQuery(ch_conn, "
  SELECT
    toStartOfWeek(received_at)   AS week_start,
    COUNT(*)                     AS order_count,
```
```

```
    SUM(order_quantity)          AS total_units
  FROM orders
  WHERE operation != 'DELETE'
  GROUP BY week_start
  ORDER BY week_start
")

weekly_orders$week_start <- as.Date(weekly_orders$week_start)

ggplot(weekly_orders, aes(x = week_start, y = order_count)) +
  geom_line(colour = "#4F6EA4", linewidth = 1.2) +
  geom_point(colour = "#E03C31", size = 2.5) +
  scale_x_date(date_labels = "%d %b", date_breaks = "2 weeks") +
  scale_y_continuous(labels = comma) +
  labs(
    title    = "Weekly Order Volume",
    subtitle = "90-day view from ClickHouse warehouse data",
    x        = "Week Starting",
    y        = "Number of Orders",
    caption  = "Source: ClickHouse - orders table"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title   = element_text(face = "bold"),
    axis.text.x  = element_text(angle = 30, hjust = 1)
  )
```



---

4 Hypothesis Tests

A data analyst does not simply describe data and create appealing visualizations. **No!** They proceed to create hypotheses and test these hypotheses. The answers to the hypothesis tests is what supports **data-driven decisions**. This section demonstrates two statistical tests using

the orders data. Subsequent labs will focus on other statistical tests of hypothesis that you need to learn how to perform.

4.1 Hypothesis Test 1: Do Bulk Orders Differ in Quantity?

Business question: Do customers who place bulk orders (quantity > 5) place significantly larger orders than customers who place standard orders? Or is the 5-unit threshold arbitrary?

Hypotheses:

- H : The mean order quantity for bulk orders equals the mean order quantity for standard orders. (_bulk = _standard)
- H : The mean order quantity for bulk orders is greater than for standard orders. (_bulk > _standard)

```
```{r t-test-bulk}
# Pull the raw quantities - we need individual records for the t-test
# rand() % 10 = 0 filters rows stochastically

# Analogy: imagine rolling a ten-sided die for every row. You keep the row only if
# it lands on one specific face. Each row gets its own independent roll, so the
# selection is random and spread across the entire table rather than concentrated at
# the top.
quantities <- dbGetQuery(ch_conn, "
  SELECT order_quantity, is_bulk_order
  FROM orders
  WHERE operation != 'DELETE'
    AND rand() % 10 = 0   -- roughly 10% random sample
  LIMIT 200000
")

bulk_qty     <- quantities$order_quantity[quantities$is_bulk_order == 1]
standard_qty <- quantities$order_quantity[quantities$is_bulk_order == 0]

cat(sprintf("Bulk orders     : n = %s, mean = %.2f, sd = %.2f\n",
    format(length(bulk_qty), big.mark=","),
    mean(bulk_qty), sd(bulk_qty)))

cat(sprintf("Standard orders : n = %s, mean = %.2f, sd = %.2f\n",
    format(length(standard_qty), big.mark=","),
    mean(standard_qty), sd(standard_qty)))
```
```

```r
# One-sided Welch's t-test (does not assume equal variances)
t_result <- t.test(
  bulk_qty,
  standard_qty,
  alternative = "greater",
  var.equal   = FALSE
)

print(t_result)

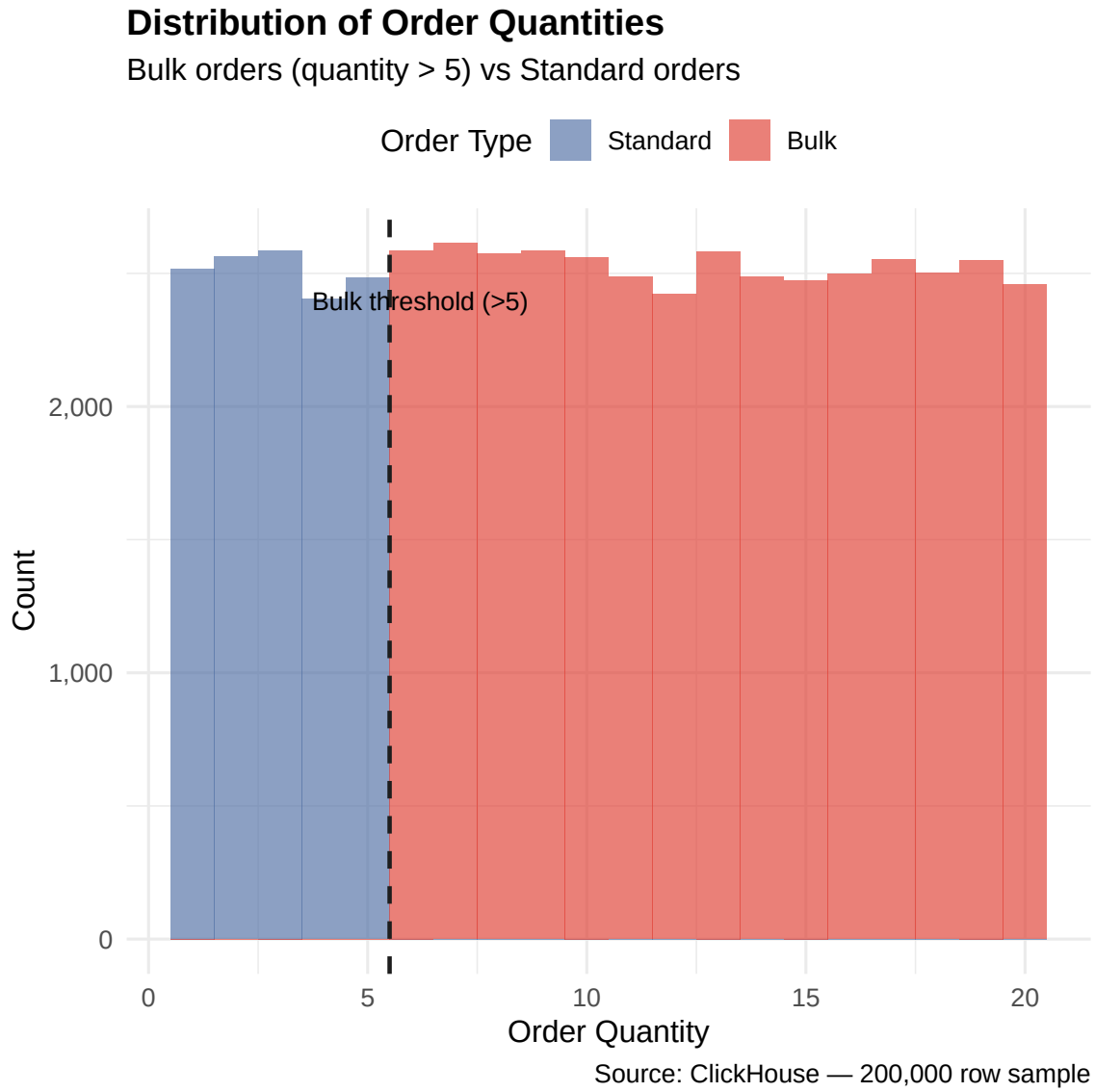
cat("\n--- INTERPRETATION ---\n")
if (t_result$p.value < 0.05) {
  cat(sprintf(
    # Note: We use %e (not e.g., %.6f) because the p-value is significantly
    # smaller than 0.0001
    "We reject H  at the 5%% significance level (p = %e).\n",
    t_result$p.value
  ))
  cat("There is statistically significant evidence that bulk orders\n")
  cat("have a higher mean quantity than standard orders.\n")
} else {
  cat(sprintf(
    "We fail to reject H  (p = %e).\n",
    t_result$p.value
  ))
  cat("There is insufficient evidence of a significant difference.\n")
}
```

```
#> Bulk orders     : n = 37,933, mean = 12.95, sd = 4.33
#> Standard orders : n = 12,557, mean = 2.98, sd = 1.41
#>
#>  Welch Two Sample t-test
#>
#> data:  bulk_qty and standard_qty
#> t = 389.93, df = 50477, p-value < 2.2e-16
#> alternative hypothesis: true difference in means is greater than 0
#> 95 percent confidence interval:
#>  9.925757      Inf
#> sample estimates:
#> mean of x mean of y
#> 12.949569  2.981763
```

```
#>
#>
#> --- INTERPRETATION ---
#> We reject H  at the 5% significance level (p = 0.000000e+00).
#> There is statistically significant evidence that bulk orders
#> have a higher mean quantity than standard orders.
```

```r
```{r ttest-visualise}
# Visualise the distribution of quantities for both groups
quantities %>%
  mutate(order_type = if_else(is_bulk_order == 1, "Bulk", "Standard"),
         order_type = factor(order_type, levels = c("Standard", "Bulk"))) %>%
  ggplot(aes(x = order_quantity, fill = order_type)) +
  # When we use `geom_histogram(binwidth = 1)`, R centers the bins over the
  # integers by default. This means:
    # The bin for a quantity of 5 actually covers the range from 4.5 to 5.5.
    # The bin for a quantity of 6 actually covers the range from 5.5 to 6.5.
  # If we were to place the xintercept exactly at 5.0, the line would cut right
  # through the middle of the "Standard" bin for 5 units.
  # By placing it at 5.5, we are drawing the line in the "no man's land" between
  # the bin for 5 and the bin for 6. This visually reinforces the logical boundary
  # that anything greater than 5 (starting at 6) is considered "Bulk."
  geom_histogram(binwidth = 1, position = "identity", alpha = 0.65) +
  geom_vline(
    xintercept = 5.5, linetype = "dashed",
    colour = "#1D1D1D", linewidth = 0.8
  ) +
  annotate("text", x = 6.2, y = Inf, vjust = 5.5,
           label = "Bulk threshold (>5)", size = 3.5) +
  scale_fill_manual(
    values = c("Standard" = "#4F6EA4", "Bulk" = "#E03C31"),
    name   = "Order Type"
  ) +
  scale_y_continuous(labels = comma) +
  labs(
    title    = "Distribution of Order Quantities",
    subtitle = "Bulk orders (quantity > 5) vs Standard orders",
    x        = "Order Quantity",
    y        = "Count",
    caption  = "Source: ClickHouse - 200,000 row sample"
  ) +
  theme_minimal(base_size = 12) +
```
```

```
  theme(
    plot.title      = element_text(face = "bold"),
    legend.position = "top"
  )
```



4.2 Hypothesis Test 2: Is Order Quantity Independent of Item Type?

Business question: Does the type of item ordered influence whether it is a bulk or standard order? Or do customers order all items in roughly equal proportions of bulk and standard?

Hypotheses:

- H : Order type (bulk vs standard) is independent of item type.
- H : There is a significant association between item type and order type.

```
```{r chisq-test}
# Build a contingency table
contingency_data <- dbGetQuery(ch_conn, "
  SELECT
    item,
    SUM(CASE WHEN is_bulk_order = 1 THEN 1 ELSE 0 END) AS bulk,
    SUM(CASE WHEN is_bulk_order = 0 THEN 1 ELSE 0 END) AS standard
  FROM orders
  WHERE operation != 'DELETE'
    AND rand() % 30 = 0   -- roughly 30% random sample
  GROUP BY item
  ORDER BY item
  LIMIT 200000
")

# Explicitly cast to integer to clear the bit64 mapping issue common in ODBC
contingency_data$bulk <- as.integer(contingency_data$bulk)
contingency_data$standard <- as.integer(contingency_data$standard)

# Disable scientific notation (e.g., 7.156541e-320) for the current session
options(scipen = 999)

# Convert to matrix for chi-square test
contingency_matrix <- as.matrix(contingency_data[, c("bulk", "standard")])
rownames(contingency_matrix) <- contingency_data$item

cat("Contingency table:\n")
print(contingency_matrix)

# Run the test
chi_result <- chisq.test(contingency_matrix)
print(chi_result)
```
```

```r
cat("\n--- INTERPRETATION ---\n")

# -----------------------
# Assumption Check: Ensure all expected frequencies are >= 5
expected <- chi_result$expected

if (any(expected < 5)) {
  warning("Some expected counts are below 5. Chi-square assumptions violated.")
}

# -----------------------
# Final Hypothesis Decision
if (is.na(chi_result$p.value)) {
  cat("Chi-square test could not compute a valid p-value.\n")
  cat("This usually happens when expected frequencies are too small\n")
  cat("or when some categories have zero counts.\n")
  cat("Consider combining categories or using Fisher's Exact Test in this case.\n")
} else if (chi_result$p.value < 0.05) {
  cat(sprintf(
    "We reject H  at the 5%% significance level (p = %e).\n",
    chi_result$p.value
  ))
  cat("There is a statistically significant association between\n")
  cat("item type and whether an order is bulk or standard.\n")
  cat("In summary: The item choice influences the likelihood of a bulk order.\n")
  cat("\nBoardroom-Ready Summary (plain language): The type of food ordered\n",
      "(e.g., Ugali vs. Managu) directly influences whether an order is\n",
      "likely to be Bulk or Standard. They are not independent; certain items\n",
      "are much more prone to being ordered in large quantities than others.")
} else {
  cat(sprintf(
    "We fail to reject H  (p = %.4f).\n",
    chi_result$p.value
  ))
  cat("Item type and order type appear to be independent.\n")
  cat("In summary: The item choice does not seem to affect order size.\n")
  cat("\nBoardroom-Ready Summary (plain language): Across our orders data,\n",
      "the type of food ordered - whether Ugali, Managu, or any other item -\n",
      "does not appear to influence whether a customer places a bulk or\n",
      "standard order. In other words, large orders are just as likely\n",
      "regardless of what is on the menu. Order size seems to be driven\n",
      "by something other than the item itself, such as the customer type,\n",
```

```
      "branch, or time of day.")
}
```

```
#> Contingency table:
#>             bulk standard
#> Egusi Soup   782      264
#> Fufu         712      257
#> Injera       710      265
#> Jollof Rice  662      241
#> Kaimati      795      256
#> Mahamri      711      240
#> Mahindi      764      250
#> Managu       772      257
#> Matoke       728      230
#> Matumbo      724      266
#> Mutura       761      267
#> Nyama Choma  734      234
#> Nyanya       675      259
#> Omena        733      268
#> Spinach      741      234
#> Sukuma Wiki  768      272
#> Ugali        735      228
#>
#>  Pearson's Chi-squared test
#>
#> data:  contingency_matrix
#> X-squared = 13.447, df = 16, p-value = 0.6398
#>
#>
#> --- INTERPRETATION ---
#> We fail to reject H  (p = 0.6398).
#> Item type and order type appear to be independent.
#> In summary: The item choice does not seem to affect order size.
#>
#> Boardroom-Ready Summary (plain language): Across our orders data,
#>  the type of food ordered - whether Ugali, Managu, or any other item -
#>  does not appear to influence whether a customer places a bulk or
#>  standard order. In other words, large orders are just as likely
#>  regardless of what is on the menu. Order size seems to be driven
#>  by something other than the item itself, such as the customer type,
#>  branch, or time of day.
```

5 The Performance Benchmark

This is the central justification for the data pipeline we built. We will now run the **same aggregation query** on PostgreSQL and on ClickHouse and measure the execution time of each.

5.1 The "Cold Start" Problem

The "Cold Start" problem in this context of database benchmarking describes **the performance lag that occurs when a system is first initialized or restarted, forcing it to fetch data from the physical disk storage rather than from high-speed memory (RAM)**.

5.1.1 The Concept: Cold versus Warm

- **Cold Start:** The database has just started. Its memory (cache) is empty. Every request for data requires a "trip" to the hard drive or SSD, which is significantly slower than RAM.
- **Warm Start:** After the first query, the database stores the results and frequently accessed data in its RAM. Subsequent queries are "warm" because the system can skip the disk trip and serve data instantly from memory.

5.1.2 Library Analogy

Imagine you are a researcher working on your final year research and you need literature from a book available in the library:

- **Cold Start:** You enter the library for the first time today. To find information on "Test Driven Development (TDD)", you must walk deep into the aisles on a specific floor, find the specific shelf, and pull out the book. This takes 10 minutes.
- **Warm Start:** You already found the book on TDD. You leave the book open on your desk as you continue to research on a similar topic about Requirements Driven Development (RDD). The next time you need to check a detail on TDD again, you simply look down at the table where you left the book. This takes 2 seconds.

5.1.3 Why it Matters for Benchmarking

If you do not restart ("Cold Start") the server (the Docker containers), PostgreSQL might appear as fast as ClickHouse simply because it is reading from the "desk" (RAM). By forcing a Cold Start, we are testing the **architectural efficiency** of the database engines—specifically how ClickHouse's columnar storage allows it to scan only the necessary data from the disk, whereas PostgreSQL's row-based structure forces it to read much more than is actually required.

5.1.4 Strategic Counter-Position

While Cold Start benchmarks reveal raw architectural power, **Warm Start** benchmarks are arguably more representative of a healthy production environment. An optimized PostgreSQL instance should ideally be "warm" 99% of the time. The real "victory" for ClickHouse is not just that it is faster on a cold start, but that its columnar compression allows it to keep **much more data** in a "warm" state using the same amount of RAM compared to PostgreSQL.

Run this in your terminal:

```
docker restart postgres clickhouse && docker ps
```

Wait until both PostgreSQL and ClickHouse are ready to receive connections. You can confirm that they are ready by connecting via DataGrip or using the client available in the each container, i.e.,

For PostgreSQL:
```
docker exec -it postgres psql -U lab_user -W -d lab_db
```

For ClickHouse:

```
docker exec -it clickhouse clickhouse-client
```

5.2 The Query

The benchmark query computes, for each item, the total number of orders, the total units ordered, the average order quantity, and the bulk order rate. This is a typical OLAP query — it scans every row, reads multiple columns, and produces a grouped aggregation.

````
```{r benchmark-query-text}
# Define the queries for each database.
# The logic is identical - only the column names differ (client_fname vs
# customer_name) because the transformer renamed the field.

PG_QUERY <- "
````
````

```
  SELECT
    item,
    COUNT(*)                                AS order_count,
    SUM(order_quantity)                     AS total_units,
    ROUND(AVG(order_quantity), 2)           AS avg_quantity,
    SUM(CASE WHEN order_quantity > 5
             THEN 1 ELSE 0 END)             AS bulk_orders,
    ROUND(((SUM(CASE WHEN order_quantity > 5 THEN 1 ELSE 0 END))::numeric / COUNT(*)) * 100,
  FROM orders
  GROUP BY orders.item
  ORDER BY total_units DESC
"

CH_QUERY <- "
  SELECT
      item,
      COUNT(*)                                AS order_count,
      SUM(order_quantity)                     AS total_units,
      ROUND(AVG(order_quantity), 2)           AS avg_quantity,
      SUM(CASE WHEN order_quantity > 5
               THEN 1 ELSE 0 END)             AS bulk_orders,
      ROUND(CAST((SUM(CASE WHEN order_quantity > 5 THEN 1 ELSE 0 END)) AS Float64) / COUNT(*)
    FROM orders
    WHERE orders.operation != 'DELETE'
    GROUP BY orders.item
    ORDER BY total_units DESC
"

cat("Both queries are defined and ready to be executed.\n")
cat("PostgreSQL query will scan:  ", format(pg_count, big.mark=","), " rows\n")
cat("ClickHouse query will scan:  ", format(ch_count, big.mark=","), " rows\n")
```

```
#> Both queries are defined and ready to be executed.
#> PostgreSQL query will scan:   503262  rows
#> ClickHouse query will scan:   503262  rows
```

5.3 Run the Benchmark

The `bench::mark()` function runs each query multiple times and reports the median, mean, and standard deviation of execution times. Using `bench` rather than `system.time()` produces

more reliable results because it controls for CPU caching and Just-In-Time (JIT) effects.

Normally, code is either **interpreted** (read line-by-line, which is slow) or **compiled** (translated into machine code all at once, which is fast). JIT is a hybrid. The system starts by interpreting the code, but as it notices certain parts are being run frequently, it compiles those specific parts into optimized machine code "on the fly" (Just-In-Time).

```
```{r benchmark-run}
# Warm up both connections with a trivial query before benchmarking.
# This ensures the connection overhead is not included in the timed results.
invisible(dbGetQuery(pg_conn, "SELECT 1"))
invisible(dbGetQuery(ch_conn, "SELECT 1"))

cat("Running benchmark - this may take 1 to 2 minutes...\n")
cat("Each query runs", 15, "times. Median time is reported.\n\n")

benchmark_results <- bench::mark(
  PostgreSQL  = dbGetQuery(pg_conn, PG_QUERY),
  ClickHouse  = dbGetQuery(ch_conn, CH_QUERY),
  iterations  = 15,
  check       = FALSE
)

# Extract median times for comparison
pg_median <- as.numeric(benchmark_results$median[1])
ch_median <- as.numeric(benchmark_results$median[2])
speedup   <- pg_median / ch_median

cat("\n===== BENCHMARK RESULTS =====\n")
cat(sprintf(
  "PostgreSQL  median : %s\n",
  format(benchmark_results$median[1])
))
cat(sprintf(
  "ClickHouse  median : %s\n",
  format(benchmark_results$median[2])
))
cat(sprintf(
  "ClickHouse was %.1f× faster than PostgreSQL\n",
  speedup
))
cat("============================\n")
```
```

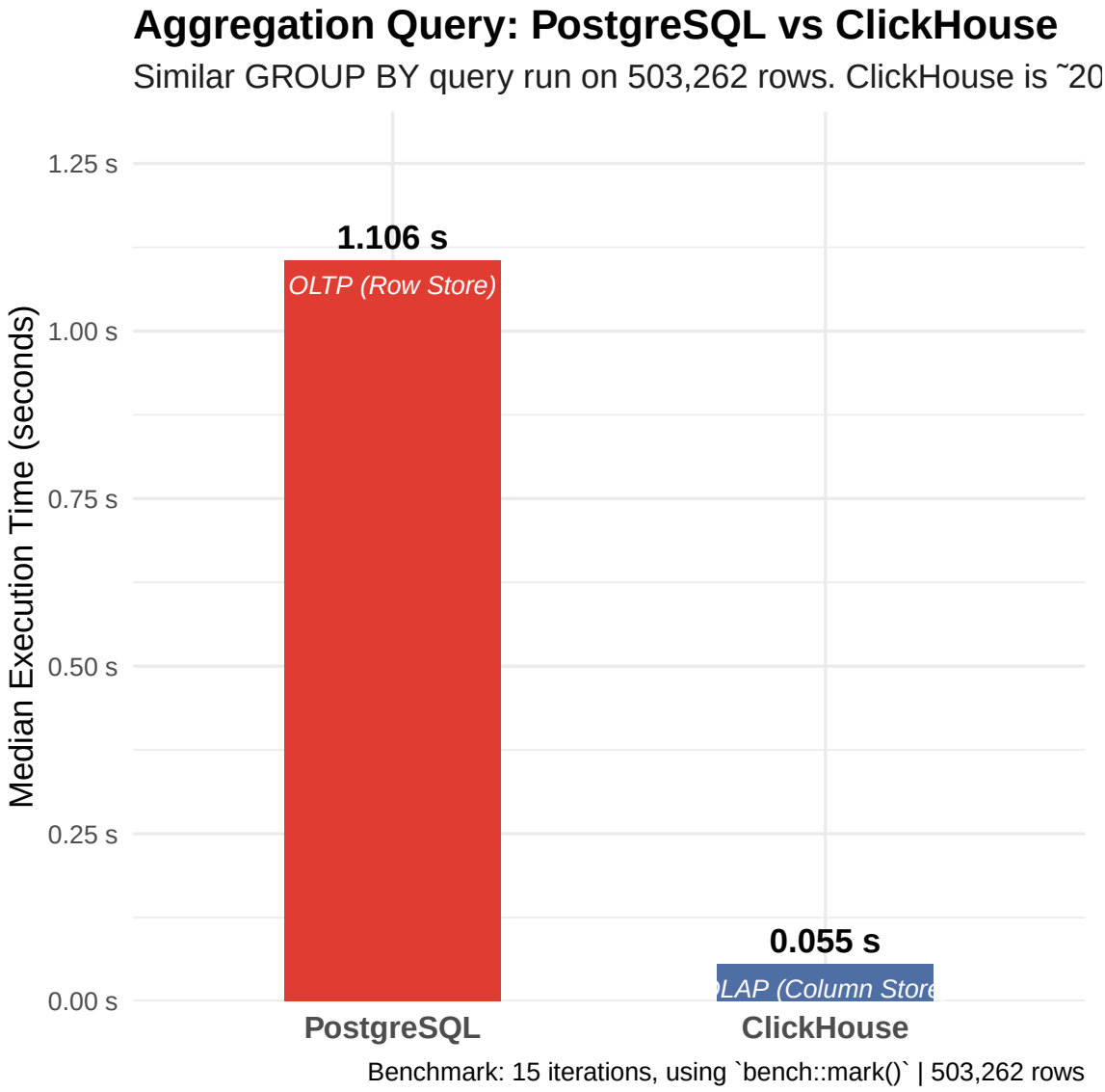
```
#> Running benchmark - this may take 1 to 2 minutes...
#> Each query runs 15 times. Median time is reported.
#>
#>
#> ===== BENCHMARK RESULTS =====
#> PostgreSQL  median : 1.11s
#> ClickHouse  median : 54.7ms
#> ClickHouse was 20.2× faster than PostgreSQL
#> ============================
```

5.4 Visualise the Results

```
```{r benchmark-visualise}
# Build a data frame from the benchmark results for plotting
bench_df <- data.frame(
  database       = c("PostgreSQL", "ClickHouse"),
  median_seconds = c(pg_median, ch_median),
  db_type        = c("OLTP (Row Store)", "OLAP (Column Store)")
)

ggplot(bench_df,
       aes(x = reorder(database, -median_seconds),
           y = median_seconds,
           fill = database)) +
  geom_col(width = 0.5, show.legend = FALSE) +
  geom_text(
    aes(label = sprintf("%.3f s", median_seconds)),
    vjust = -0.5, size = 4.5, fontface = "bold"
  ) +
  geom_text(
    aes(label = db_type),
    vjust = 1.8, size = 3.5, colour = "white", fontface = "italic"
  ) +
  scale_fill_manual(values = c(
    "PostgreSQL"  = "#E03C31",
    "ClickHouse"  = "#4F6EA4"
  )) +
  scale_y_continuous(
    labels   = label_number(suffix = " s"),
    expand   = expansion(mult = c(0, 0.2))
  ) +
```
```

```
  labs(
    title    = "Aggregation Query: PostgreSQL vs ClickHouse",
    subtitle = sprintf(
      "Similar GROUP BY query run on %s rows. ClickHouse is  %.1f× faster",
      format(as.numeric(max(pg_count, ch_count)), big.mark = ","),
      speedup
    ),
    x        = NULL,
    y        = "Median Execution Time (seconds)",
    caption  = paste(
      "Benchmark: 15 iterations, using `bench::mark()`",
      sprintf("| %s rows",
              format(as.numeric(max(pg_count, ch_count)), big.mark = ","))
    )
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title    = element_text(face = "bold"),
    plot.subtitle = element_text(colour = "#1D1D1D"),
    axis.text.x   = element_text(size = 12, face = "bold")
  )
```



5.5 Examine the Query Results

Both queries return a similar business answer — only via different paths.

```
```{r benchmark-results-table-from-PostgreSQL}
pg_result <- dbGetQuery(pg_conn, PG_QUERY)
ch_result <- dbGetQuery(ch_conn, CH_QUERY)

knitr::kable(
```
```

```
  pg_result,
  caption   = "Aggregation Result from PostgreSQL",
  col.names = c("Item", "Order Count", "Total Units",
                "Avg Quantity", "Bulk Orders", "Bulk %"),
  format.args = list(big.mark = ",")
)
```

Table 1: Aggregation Result from PostgreSQL

Item	Order Count	Total Units	Avg Quantity	Bulk Orders	Bulk %
Mahindi	29920	313276	10.47	22461	75.07
Nyama Choma	29874	311965	10.44	22242	74.45
Matumbo	29629	311815	10.52	22222	75.00
Ugali	29830	311645	10.45	22354	74.94
Sukuma Wiki	29701	311565	10.49	22232	74.85
Kaimati	29644	310761	10.48	22135	74.67
Egusi Soup	29656	310722	10.48	22185	74.81
Jollof Rice	29585	310324	10.49	22176	74.96
Spinach	29625	309933	10.46	22145	74.75
Injera	29714	309404	10.41	22123	74.45
Managu	29578	309012	10.45	22235	75.17
Omena	29422	308745	10.49	22003	74.78
Matoke	29331	308413	10.51	22032	75.12
Mahamri	29467	308317	10.46	22007	74.68
Fufu	29575	307852	10.41	22071	74.63
Mutura	29473	307824	10.44	22019	74.71
Nyanya	29238	305152	10.44	21846	74.72

```{r benchmark-results-table-from-ClickHouse}
knitr::kable(
  ch_result,
  caption   = "Aggregation Result from ClickHouse",
  col.names = c("Item", "Order Count", "Total Units",
                "Avg Quantity", "Bulk Orders", "Bulk %"),
  format.args = list(big.mark = ",")
)
```

Table 2: Aggregation Result from ClickHouse

Item	Order Count	Total Units	Avg Quantity	Bulk Orders	Bulk %
Mahindi	29920	313276	10.47	22461	75.07
Nyama Choma	29874	311965	10.44	22242	74.45
Matumbo	29629	311815	10.52	22222	75.00
Ugali	29830	311645	10.45	22354	74.94
Sukuma Wiki	29701	311565	10.49	22232	74.85
Kaimati	29644	310761	10.48	22135	74.67
Egusi Soup	29656	310722	10.48	22185	74.81
Jollof Rice	29585	310324	10.49	22176	74.96
Spinach	29625	309933	10.46	22145	74.75
Injera	29714	309404	10.41	22123	74.45
Managu	29578	309012	10.45	22235	75.17
Omena	29422	308745	10.49	22003	74.78
Matoke	29331	308413	10.51	22032	75.12
Mahamri	29467	308317	10.46	22007	74.68
Fufu	29575	307852	10.41	22071	74.63
Mutura	29473	307824	10.44	22019	74.71
Nyanya	29238	305152	10.44	21846	74.72

5.6 Why the Gap Exists: Column Store vs Row Store

```
```{r architecture-explanation}
cat(
"HOW POSTGRESQL EXECUTES THIS QUERY\n",
"                                    \n",
"1. Reads every row from disk (all columns: order_id, client_fname,\n",
"   item, order_quantity, received_at).\n",
"2. For each row, extracts only item and order_quantity.\n",
"3. Groups the extracted values and computes the aggregation.\n",
"4. The full row width is read even though only 2 of 5 columns are needed.\n",
"\n",
"HOW CLICKHOUSE EXECUTES THIS QUERY\n",
"                                    \n",
"1. Reads ONLY the columns referenced in the query:\n",
"   item, order_quantity, is_bulk_order.\n",
"2. These columns are stored contiguously on disk - no row crossing.\n",
"3. ClickHouse also compresses each column independently (similar values\n",
```
```

```
"   compress extremely well - e.g., repeated item names).\n",
"4. The grouped aggregation runs on the compressed column data directly.\n",
"\n",
"The ratio you measured is not an optimisation trick or a lucky cache hit.\n",
"It is a structural consequence of how the data is physically laid out on\n",
"disk. At 10 million rows this gap widens. At 1 billion rows it becomes\n",
"the difference between a 3-second dashboard refresh and a 45-minute wait.\n",
sep = ""
)
```

```
#> HOW POSTGRESQL EXECUTES THIS QUERY
#>
#> 1. Reads every row from disk (all columns: order_id, client_fname,
#>    item, order_quantity, received_at).
#> 2. For each row, extracts only item and order_quantity.
#> 3. Groups the extracted values and computes the aggregation.
#> 4. The full row width is read even though only 2 of 5 columns are needed.
#>
#> HOW CLICKHOUSE EXECUTES THIS QUERY
#>
#> 1. Reads ONLY the columns referenced in the query:
#>    item, order_quantity, is_bulk_order.
#> 2. These columns are stored contiguously on disk - no row crossing.
#> 3. ClickHouse also compresses each column independently (similar values
#>    compress extremely well - e.g., repeated item names).
#> 4. The grouped aggregation runs on the compressed column data directly.
#>
#> The ratio you measured is not an optimisation trick or a lucky cache hit.
#> It is a structural consequence of how the data is physically laid out on
#> disk. At 10 million rows this gap widens. At 1 billion rows it becomes
#> the difference between a 3-second dashboard refresh and a 45-minute wait.
```

---

6 Data Pipeline Latency Analysis

One of the audit fields the transformer added (feature engineering) is `processed_at` — the Nairobi time at which the transformer consumed the Kafka message and wrote the record to ClickHouse. The gap between `received_at` and `processed_at` is the end-to-end pipeline latency.

```{r latency-raw}
latency_data <- dbGetQuery(ch_conn, "
  SELECT
    item,
    operation,
    AVG(
      dateDiff('second', received_at, processed_at)
    ) AS avg_latency_seconds,
    MAX(
      dateDiff('second', received_at, processed_at)
    ) AS max_latency_seconds,
    COUNT(*) AS sample_size
  FROM orders
  WHERE operation IN ('INSERT', 'SNAPSHOT')
    AND processed_at > received_at
  GROUP BY item, operation
  ORDER BY avg_latency_seconds DESC
")

knitr::kable(
  latency_data,
  caption   = "Average Pipeline Latency by Item and Operation Type",
  col.names = c("Item", "Operation", "Avg Latency (s)",
                "Max Latency (s)", "Sample Size"),
  format.args = list(big.mark = ","),
  digits = 2
)
```

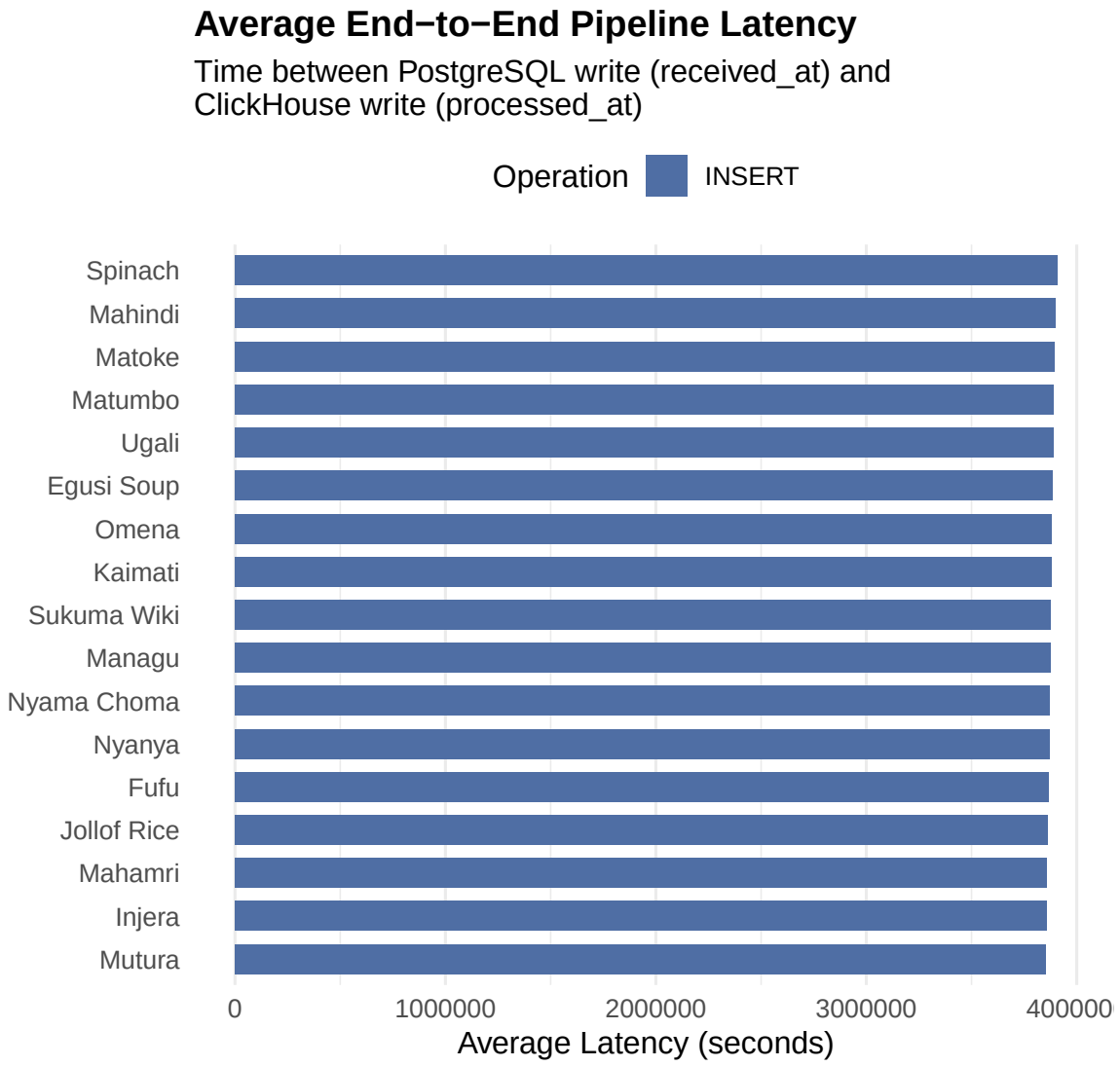
Table 3: Average Pipeline Latency by Item and Operation Type

Item	Operation	Avg Latency (s)	Max Latency (s)	Sample Size
Spinach	INSERT	3,906,575.52	7776312	29539
Mahindi	INSERT	3,897,124.29	7776398	29845
Matoke	INSERT	3,896,349.92	7775822	29234
Matumbo	INSERT	3,889,267.74	7775324	29559
Ugali	INSERT	3,887,932.21	7776614	29747
Egusi Soup	INSERT	3,883,625.09	7776262	29568
Omena	INSERT	3,882,955.55	7776235	29311
Kaimati	INSERT	3,880,724.46	7775427	29572
Sukuma Wiki	INSERT	3,877,189.58	7776383	29610

Item	Operation	Avg Latency (s)	Max Latency (s)	Sample Size
Managu	INSERT	3,875,074.89	7776263	29497
Nyama Choma	INSERT	3,873,355.67	7776616	29773
Nyanya	INSERT	3,870,275.08	7776316	29158
Fufu	INSERT	3,868,943.99	7776679	29479
Jollof Rice	INSERT	3,860,679.23	7776640	29481
Mahamri	INSERT	3,859,359.78	7776651	29388
Injera	INSERT	3,858,079.71	7776344	29623
Mutura	INSERT	3,852,737.22	7776468	29370
Mahamri	SNAPSHOT	366.33	419	3
Matumbo	SNAPSHOT	336.80	420	5
Matoke	SNAPSHOT	277.00	419	7
Egusi Soup	SNAPSHOT	273.12	419	8
Fufu	SNAPSHOT	265.89	420	9
Kaimati	SNAPSHOT	260.33	419	3
Sukuma Wiki	SNAPSHOT	255.45	420	11
Injera	SNAPSHOT	238.75	399	4
Ugali	SNAPSHOT	232.00	265	4
Nyama Choma	SNAPSHOT	225.30	419	10
Mutura	SNAPSHOT	217.50	250	2
Omena	SNAPSHOT	199.78	374	9
Nyanya	SNAPSHOT	173.67	406	3
Mahindi	SNAPSHOT	167.67	300	6
Jollof Rice	SNAPSHOT	165.00	356	3
Managu	SNAPSHOT	133.00	189	3
Spinach	SNAPSHOT	132.50	233	2

````
```{r latency-visualise}
latency_data %>%
  filter(sample_size > 100) %>%
  ggplot(aes(x = reorder(item, avg_latency_seconds),
             y = avg_latency_seconds,
             fill = operation)) +
  geom_col(position = "dodge", width = 0.7) +
  coord_flip() +
  scale_fill_manual(
    values = c("INSERT" = "#4F6EA4", "SNAPSHOT" = "#CEA152"),
    name   = "Operation"
  ) +
  labs(
    title    = "Average End-to-End Pipeline Latency",
````
````

```
    subtitle = "Time between PostgreSQL write (received_at) and\nClickHouse write (processed_
    x        = NULL,
    y        = "Average Latency (seconds)",
    caption  = "Source: ClickHouse - orders table | Only items with >100 records shown"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title         = element_text(face = "bold"),
    panel.grid.major.y = element_blank(),
    legend.position    = "top"
  )
```



Interpretation: Snapshot records (op = 'r') typically show higher latency than live inserts because the snapshot runs once at connector stars and processes a large batch of existing records. Live inserts flow through the pipeline in near real-time. A latency of under 5 seconds from PostgreSQL write to ClickHouse availability is consistent with a well-functioning CDC pipeline.

---

7 Conclusion

This notebook was used to demonstrated three things:

1. A data warehouse designed using a non-relational data model (ClickHouse) enables analytical queries that a database designed using a relational (object relational) data model (PostgreSQL) cannot serve at scale. The benchmark showed a clear performance difference on the same query against the same data volume. This difference grows with scale.

2. Statistical analysis answers real business questions. The t-test confirmed that bulk orders genuinely differ in quantity from standard orders. The chi-square test revealed whether item type influences ordering behaviour. These are questions a data analyst would ask.

3. The pipeline latency is measurable and meaningful. The `processed_at` audit field, added by the transformer, enables pipeline observability — you can measure how long it takes for a PostgreSQL change to appear in the warehouse.

The full architecture you have built across Part 1 through 4 — from Kafka producer to ClickHouse analytics — is a simplified but architecturally faithful representation of how data platforms are built in production environments.

---

8 Clean Up

Always close database connections when you are finished.

```
```{r cleanup}
dbDisconnect(ch_conn)
dbDisconnect(pg_conn)

cat("  Both connections closed.\n")
confirm_connection(ch_conn, "ch_conn (ClickHouse)")
confirm_connection(pg_conn, "pg_conn (PostgreSQL)")
```
```

```
#>   Both connections closed.
#> ch_conn (ClickHouse) -> NOT responding
#> pg_conn (PostgreSQL) -> NOT responding
```